

# Sorting

## 2. External Sorting:

-The data that is to be sorted cannot be accommodated in the memory at the same time and require some additional auxiliary memory, then it is called external sorting.

```
arr = {1,2,6,9,3,5,8};
```

Stable and Not stable sorting:

-----  
arr = {1,2,6,9,3,5,8,6,9,3,5,8};

arr = {1,2,3,3,5,5,6,6,8,8,9,9};

Stable : Sorting

or

arr = {1,2,6,9,3,5,8,6,9,3,5,8};

arr = {1,2,3,3,5,5,6,6,8,8,9,9};

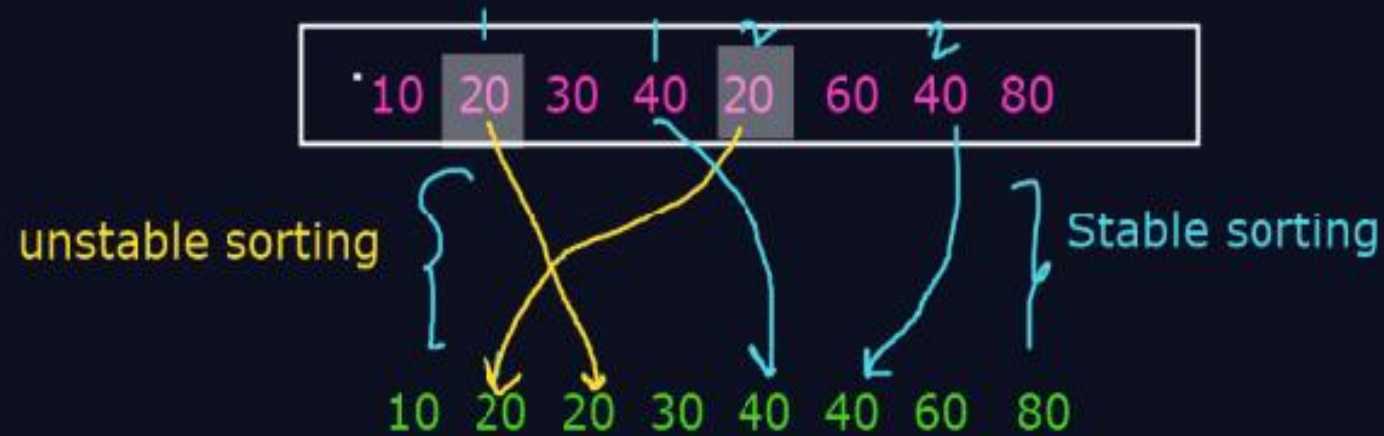
Not stable : sorting

Stable sorting:

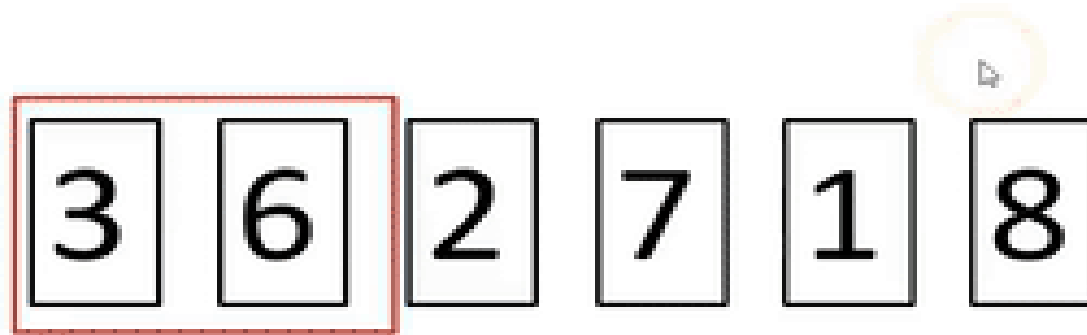
-----  
-Maintains the relative order of same (duplicate) elements after sorting.

Unstable sorting:

-----  
-Does not guarantee the relative order of equal elements after sorting.

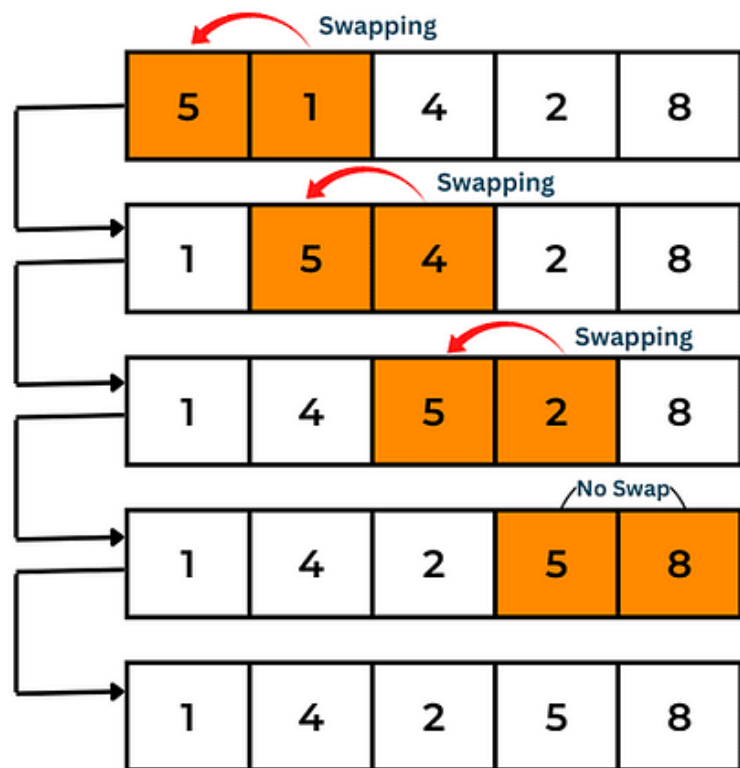


## Bubble Sort:



# BUBBLE SORTING

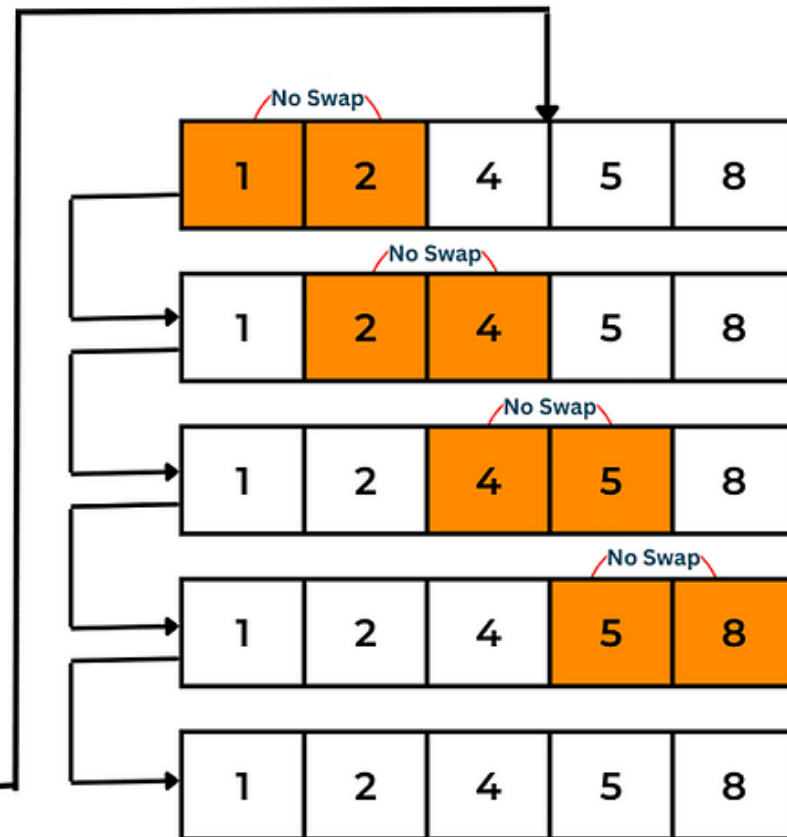
## First Pass



## Second Pass



## Third Pass



---

**Algorithm 1:** Bubble sort

---

**Data:** Input array  $A[]$

**Result:** Sorted  $A[]$

*int*  $i, j, k$ ;

$N = \text{length}(A)$ ;

**for**  $j = 1$  **to**  $N$  **do**

**for**  $i = 0$  **to**  $N-1$  **do**

**if**  $A[i] > A[i+1]$  **then**

$temp = A[i]$ ;

$A[i] = A[i+1]$ ;

$A[i+1] = temp$ ;

**end**

**end**

**end**

---

```

class BSort{

    void bSort(int arr[]){
        int n = arr.length;

        for(int i=0;i<n-1;i++){
            for(int j=0;j<n-1-1;j++){
                if(arr[j] > arr[j+1]){
                    int temp = arr[j];
                    arr[j] = arr[j+1];
                    arr[j+1] = temp;
                }
            }
        }
    }

    void display(int arr[]){
        int n = arr.length;

        for(int i=0;i<n;i++){
            System.out.print(arr[i]+ " ");
        }
        System.out.println();
    }

    public static void main(String[] args)

```

Number of elements: n

Number of comparisons: (n-1) X n

=O(n<sup>2</sup>)

Time Complexity:

-----

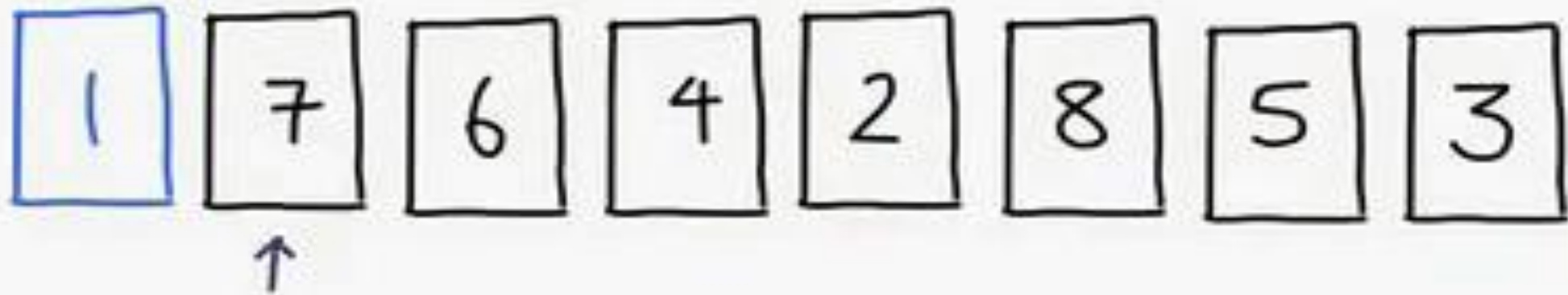
Best case : O(n<sup>2</sup>)

Average cas : O(n<sup>2</sup>)

Worst case : O(n<sup>2</sup>)

Space Complexity: O(n)

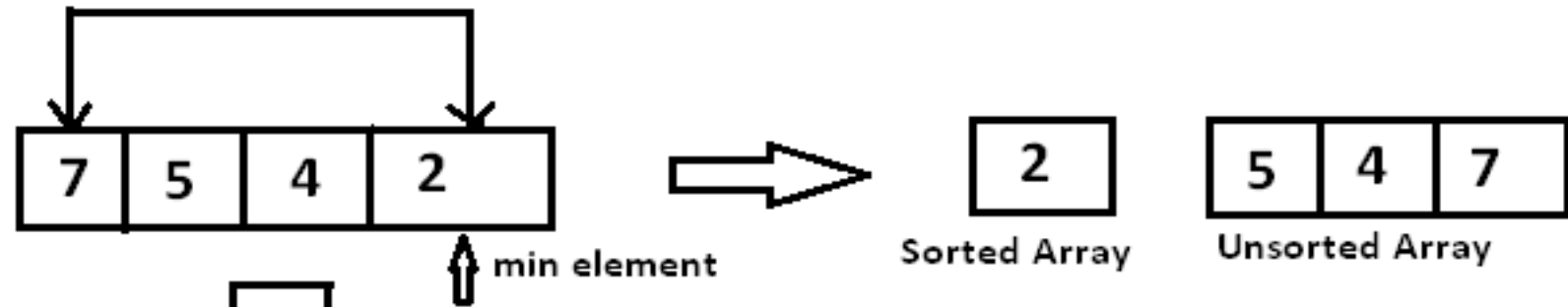
\_\_\_\_\_



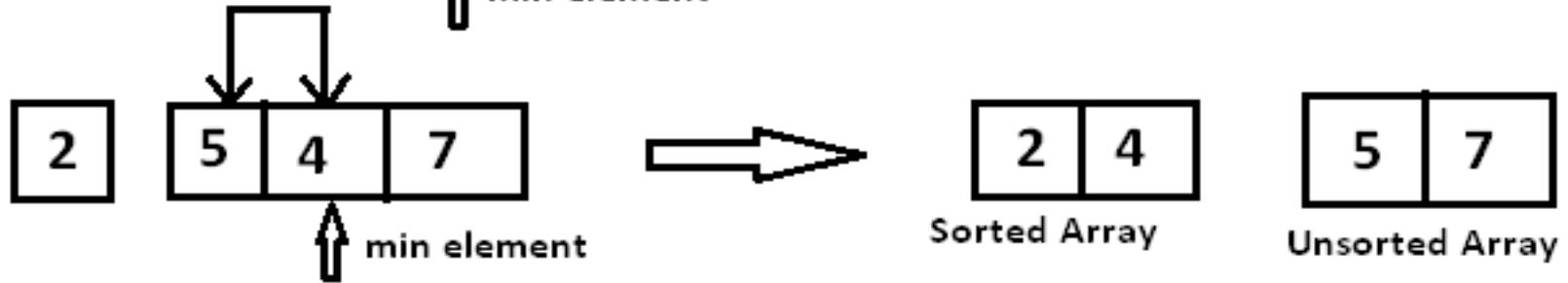
Minimum : 7



STEP 1.



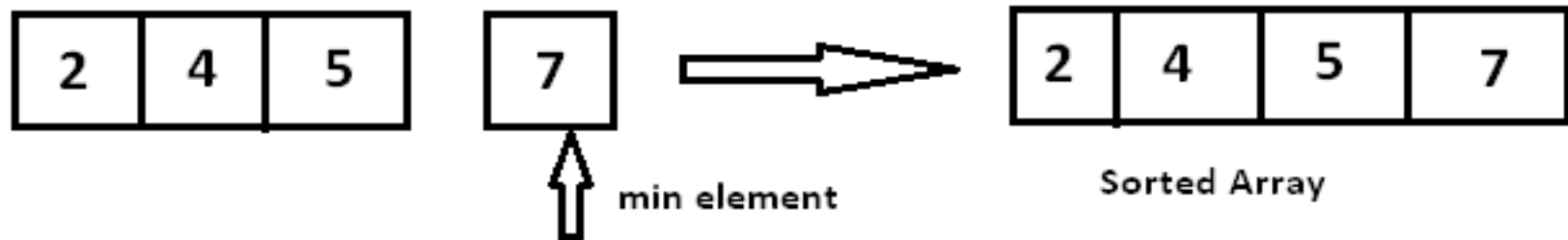
STEP 2.



STEP 3.



STEP 4.



## **Algorithm:**

**SelectionSort(A)**

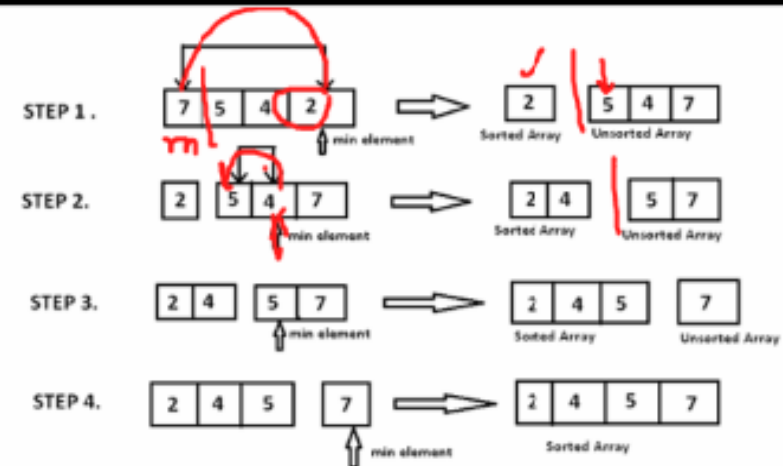
```
{
    for( i = 0; i < n ; i++)
    {
        least=A[i];
        p=i;
        for ( j = i + 1; j < n ; j++)
        {
            if (A[j] < A[i])
                least= A[j]; p=j;
        }
    }
    swap(A[i],A[p]);
}
```

```

}

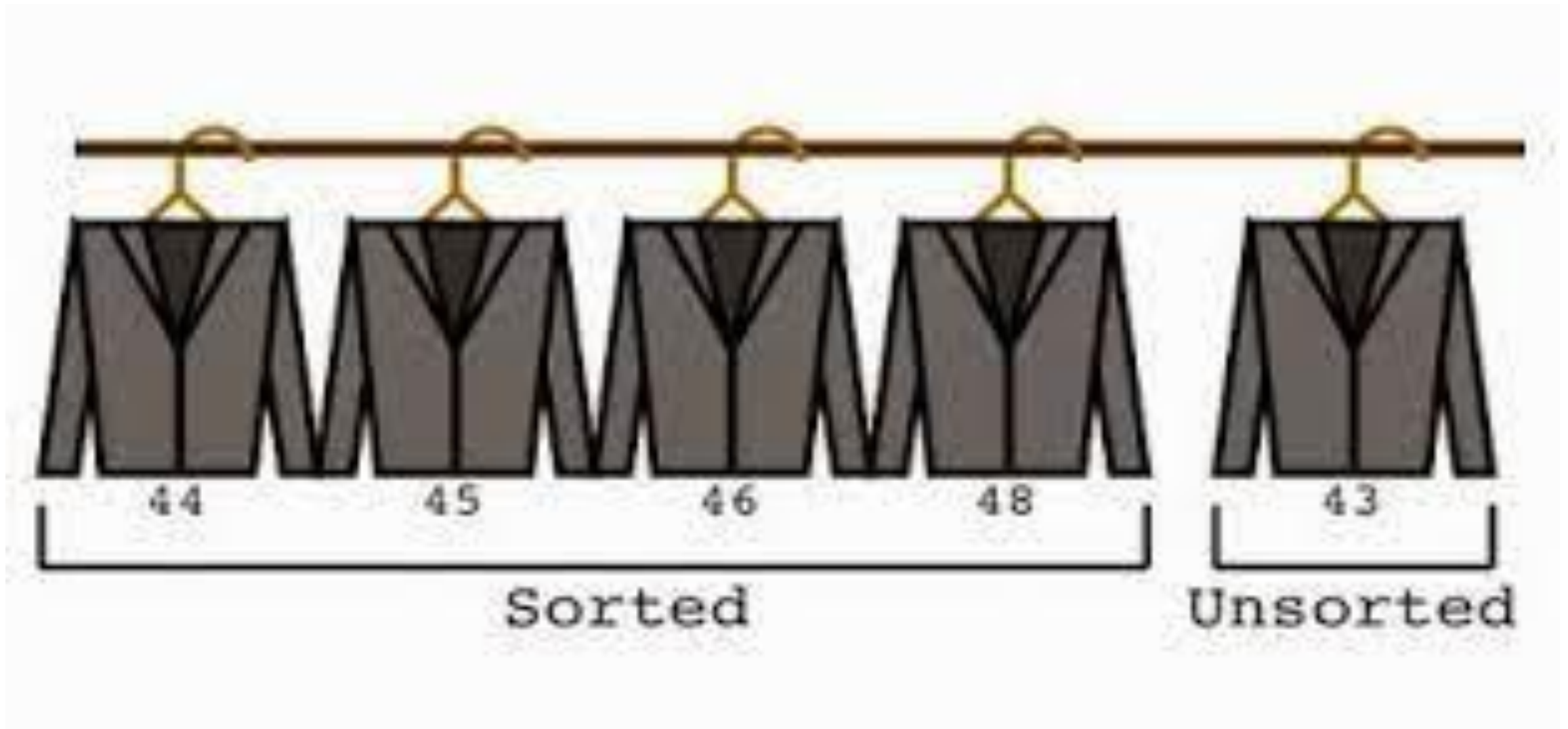
void ssort(int arr[]){
    int n = arr.length;
    for(int i=0;i<n-1;i++){
        int min = i;
        for(int j=i+1;j<n;j++){
            if(arr[j] < arr[min])
                min = j;
        }
        int temp = arr[min];
        arr[min] = arr[i];
        arr[i] = temp;
    }
}

void display(int arr[]){
    int n = arr.length;
    for(int i=0;i<n;i++){
        System.out.print(arr[i]+" ");
    }
}
    
```

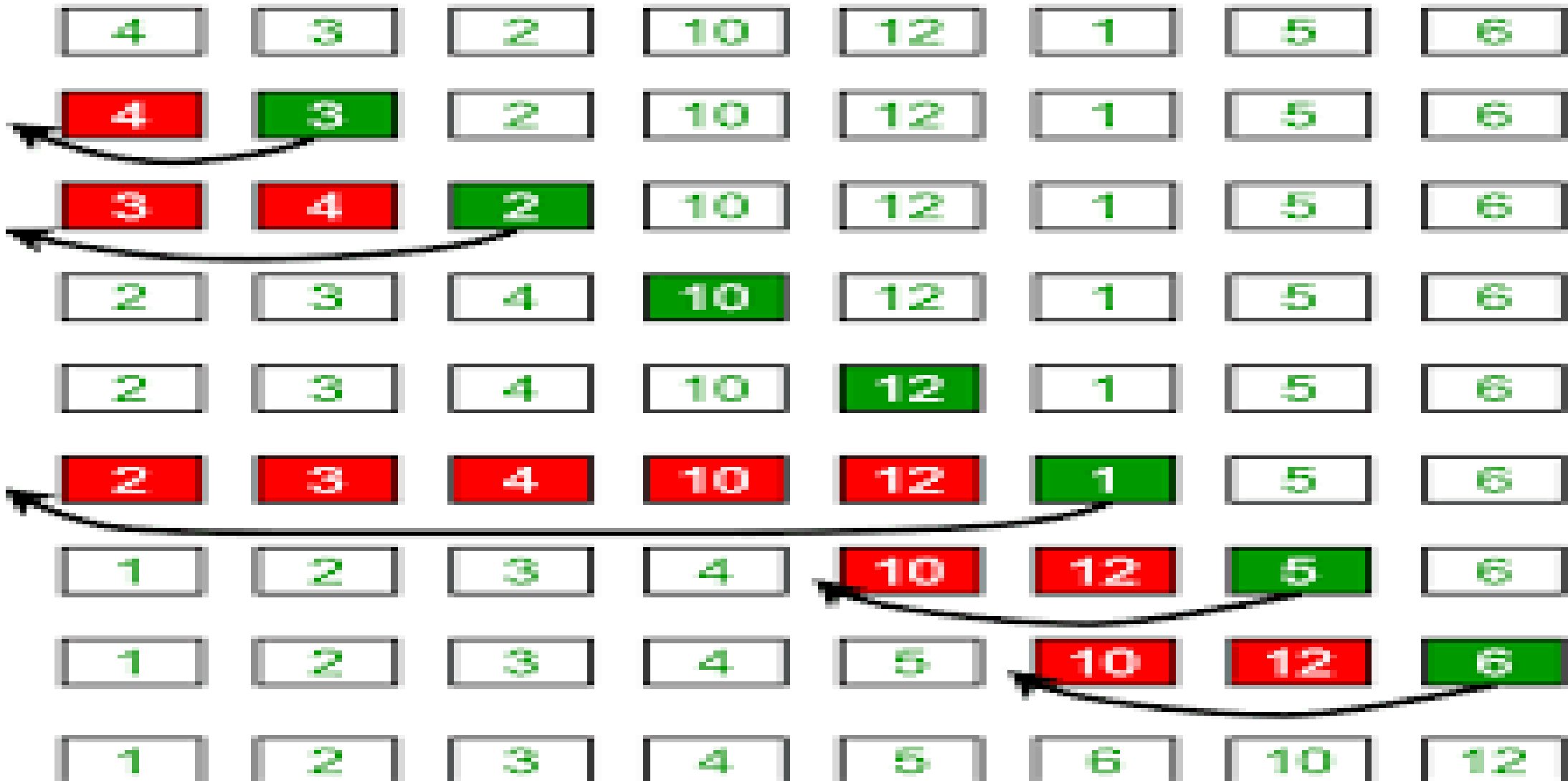


CDAC MUMBAI - Kiran Waghmare

5



## Insertion Sort Execution Example



## INSERTION-SORT(A)

for  $j \leftarrow 2$  to  $n$

do  $\text{key} \leftarrow A[j]$

▷ Insert  $A[j]$  into the sorted sequence  $A[1 \dots j-1]$

$i \leftarrow j - 1$

while  $i > 0$  and  $A[i] > \text{key}$

do  $A[i + 1] \leftarrow A[i]$

$i \leftarrow i - 1$

$A[i + 1] \leftarrow \text{key}$

cost

times

$c_1$

$n$

$c_2$

$n-1$

0

$n-1$

$c_4$

$n-1$

$c_5$

$\sum_{j=2}^n t_j$

$c_6$

$\sum_{j=2}^n (t_j - 1)$

$c_7$

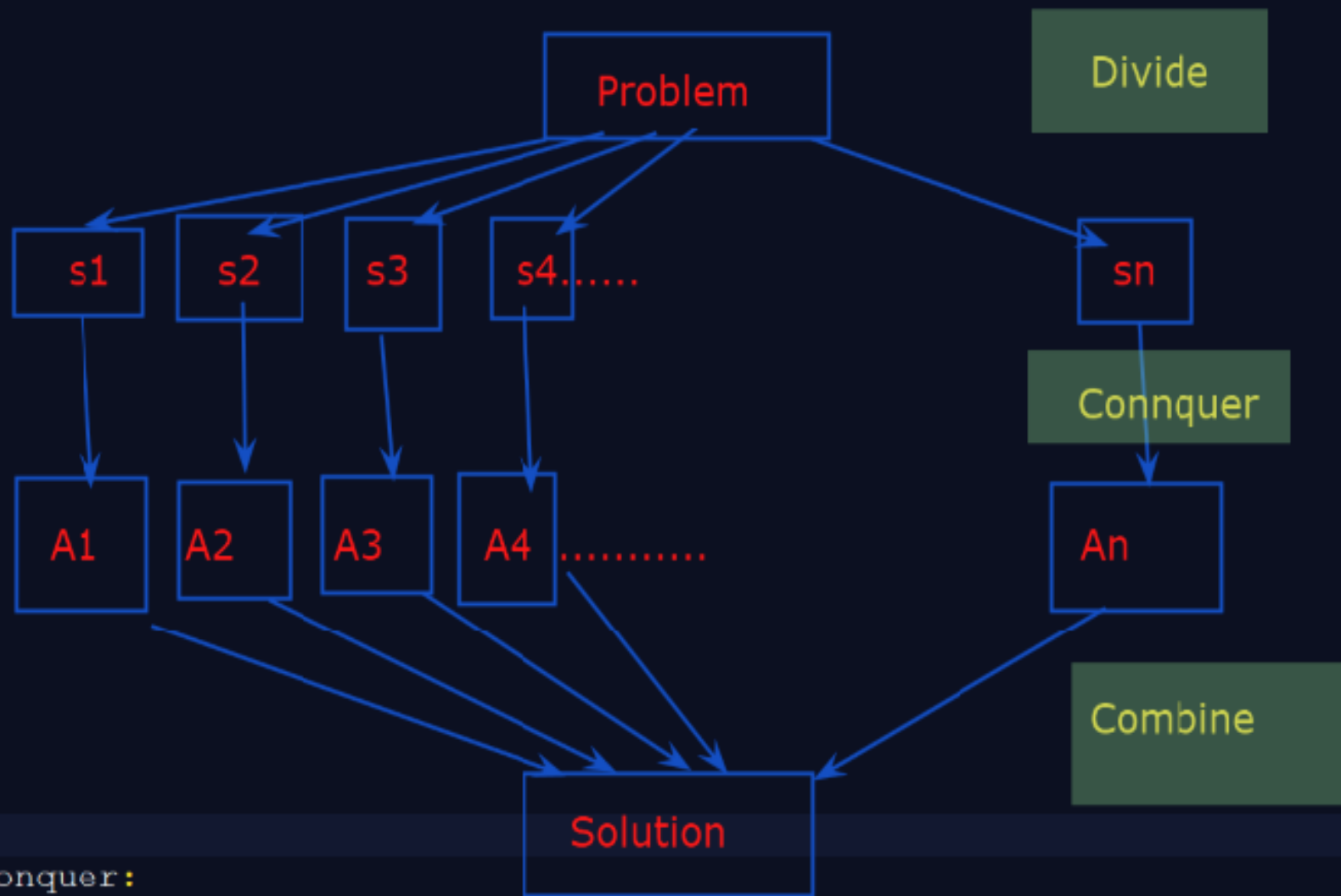
$\sum_{j=2}^n (t_j - 1)$

$c_8$

$n-1$

$t_j$ : # of times the while statement is executed at iteration  $j$

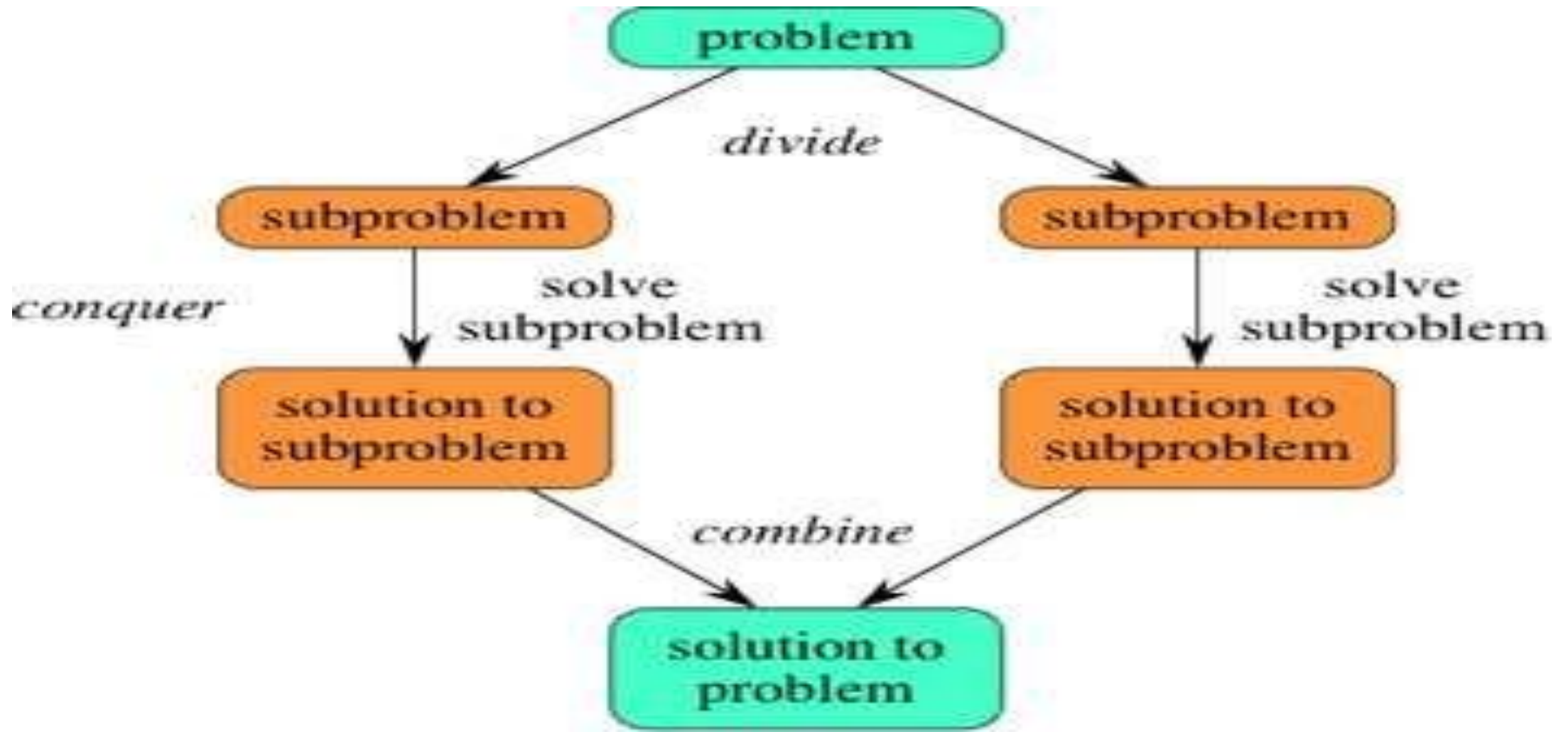
$$T(n) = c_1 n + c_2 (n-1) + c_4 (n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8 (n-1)$$



Divide and Conquer:

-----  
-It breaks down the problem into similar sub problem, that means the problem of similar origin, which recursively solves the problem and finally combines the solution.

# Divide-and-conquer





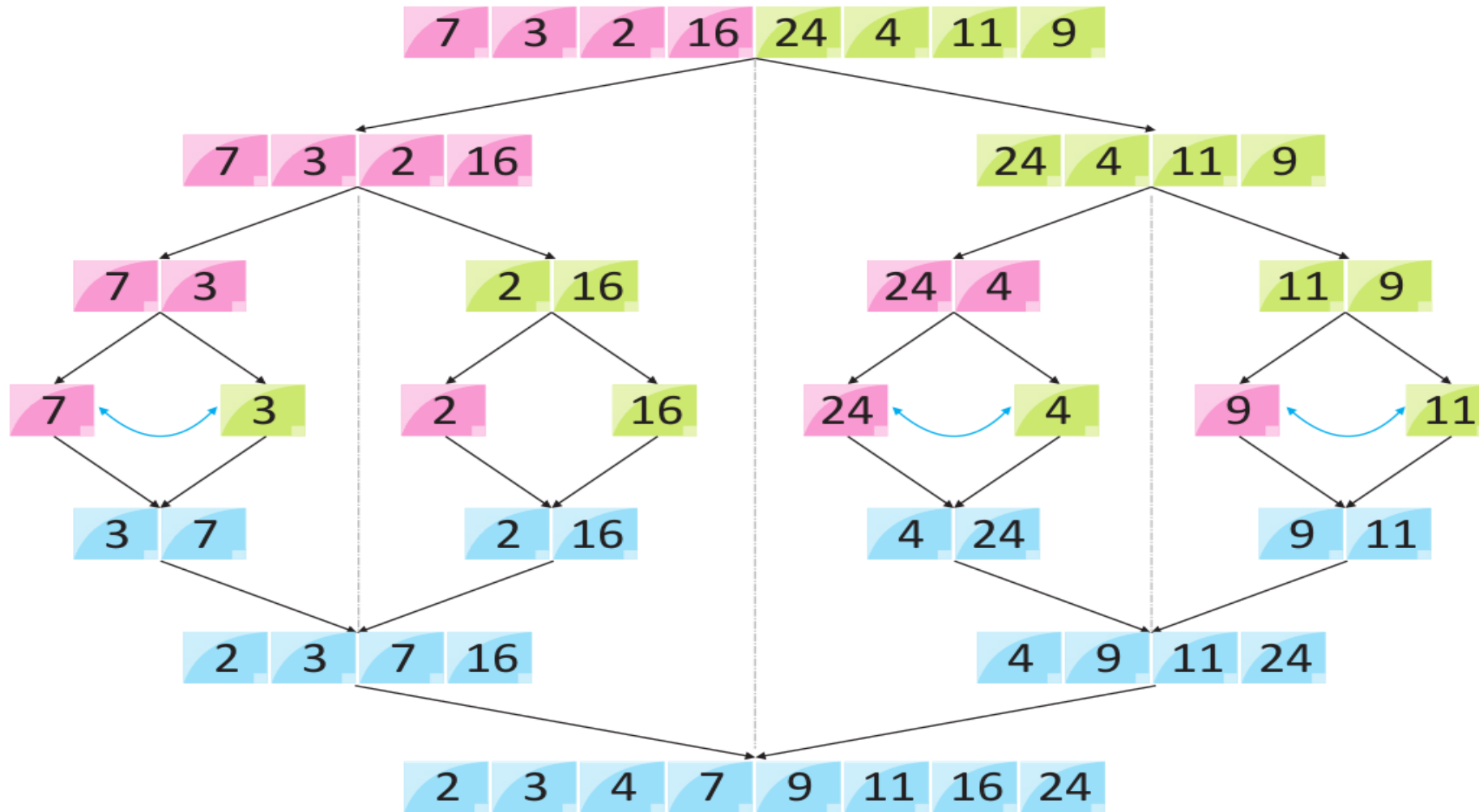
# Merge Sort:

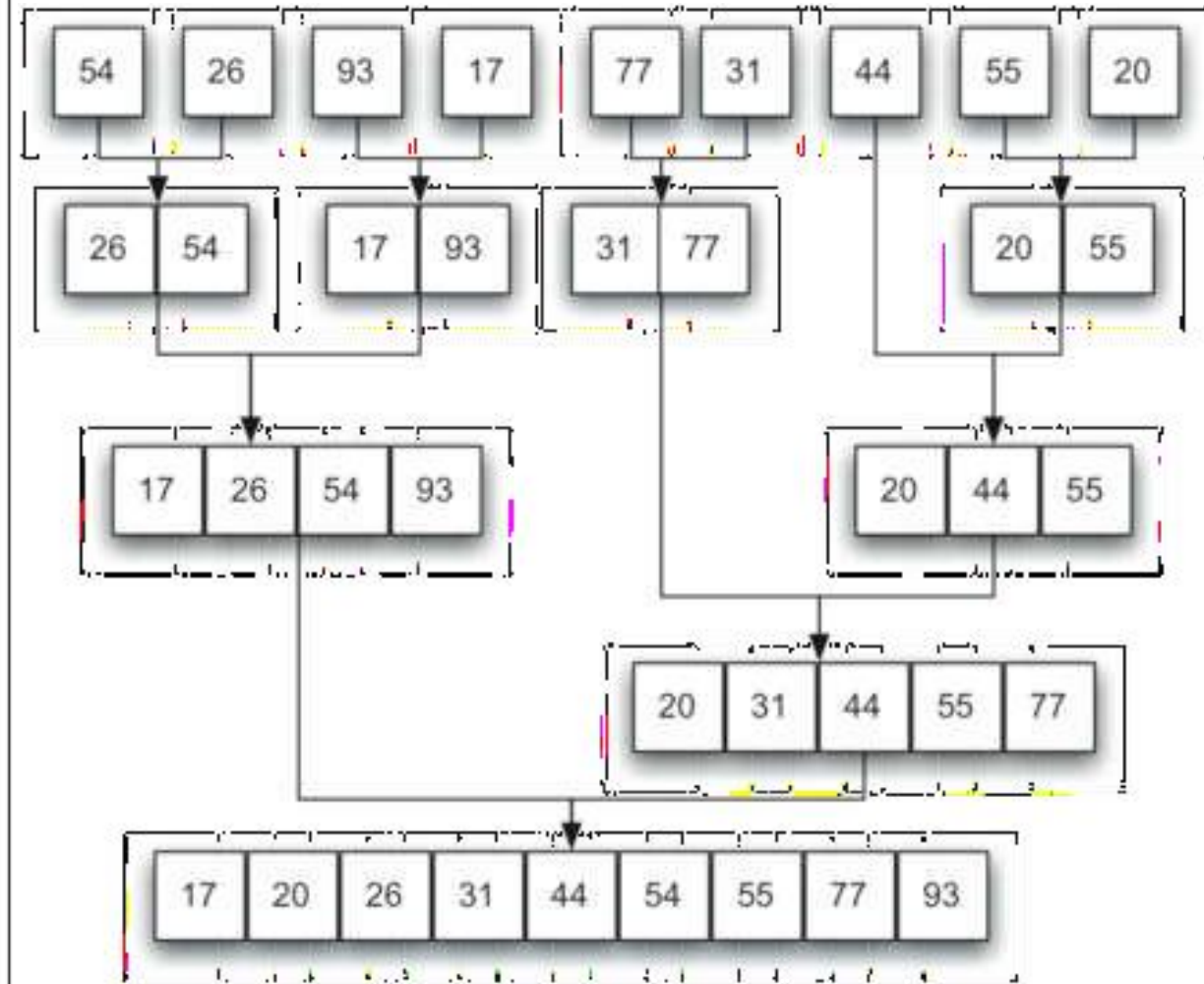
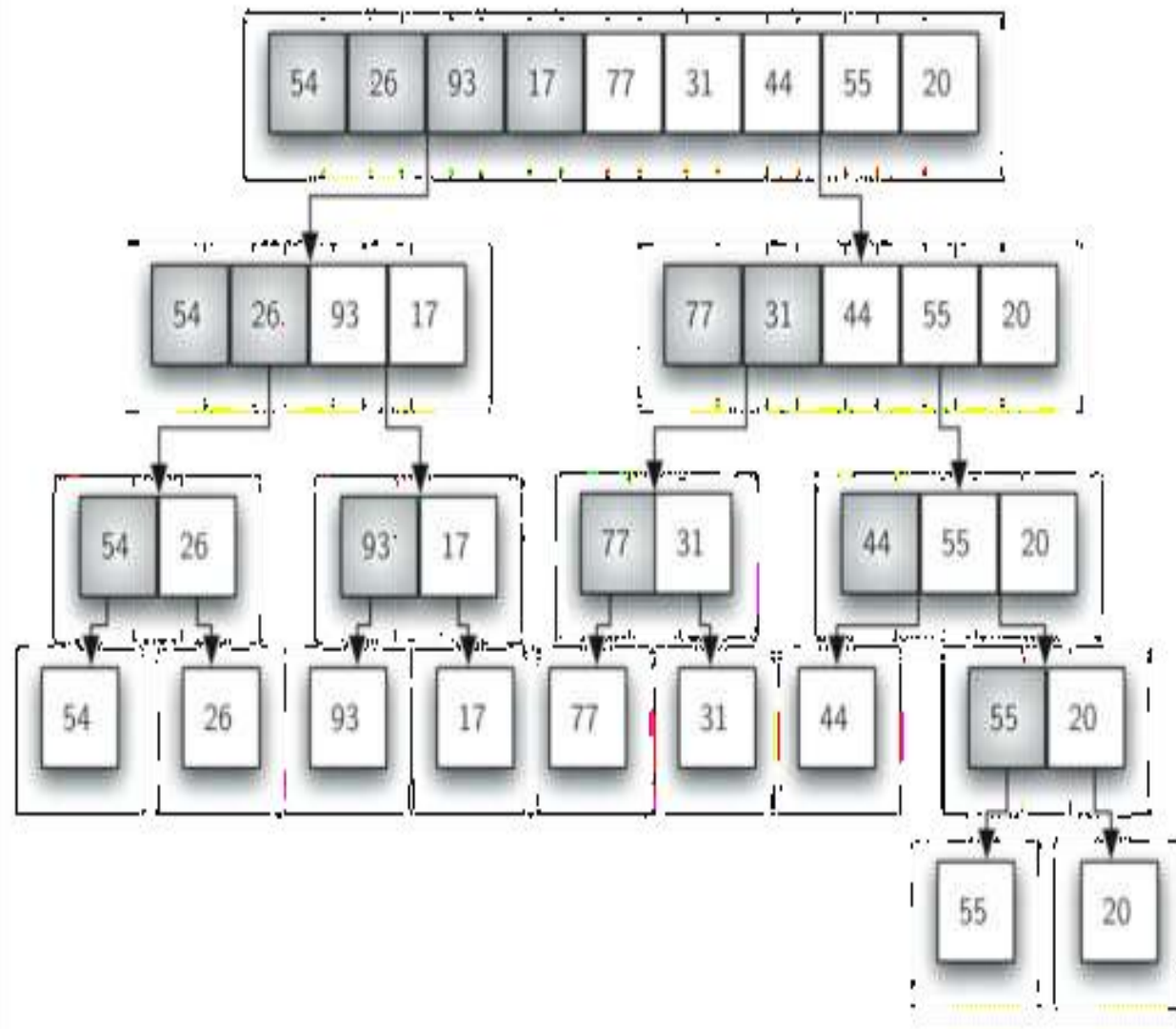
3 1 6 8 4 5 7 2

method call

return value

merge  
↓





### *Merge Sort:*

Here is the pseudocode for Merge Sort, modified to include a counter:

```
count ← 0
Merge_Sort(A, p, r)
1   if p < r
2       then q ← ⌊(p + r)/2⌋
3           Merge-Sort (A, p, q)
4           Merge-Sort (A, q+1, r)
5           Merge (A, p, q, r)
```

And here is the modified algorithm for the Merge function used by Merge Sort:

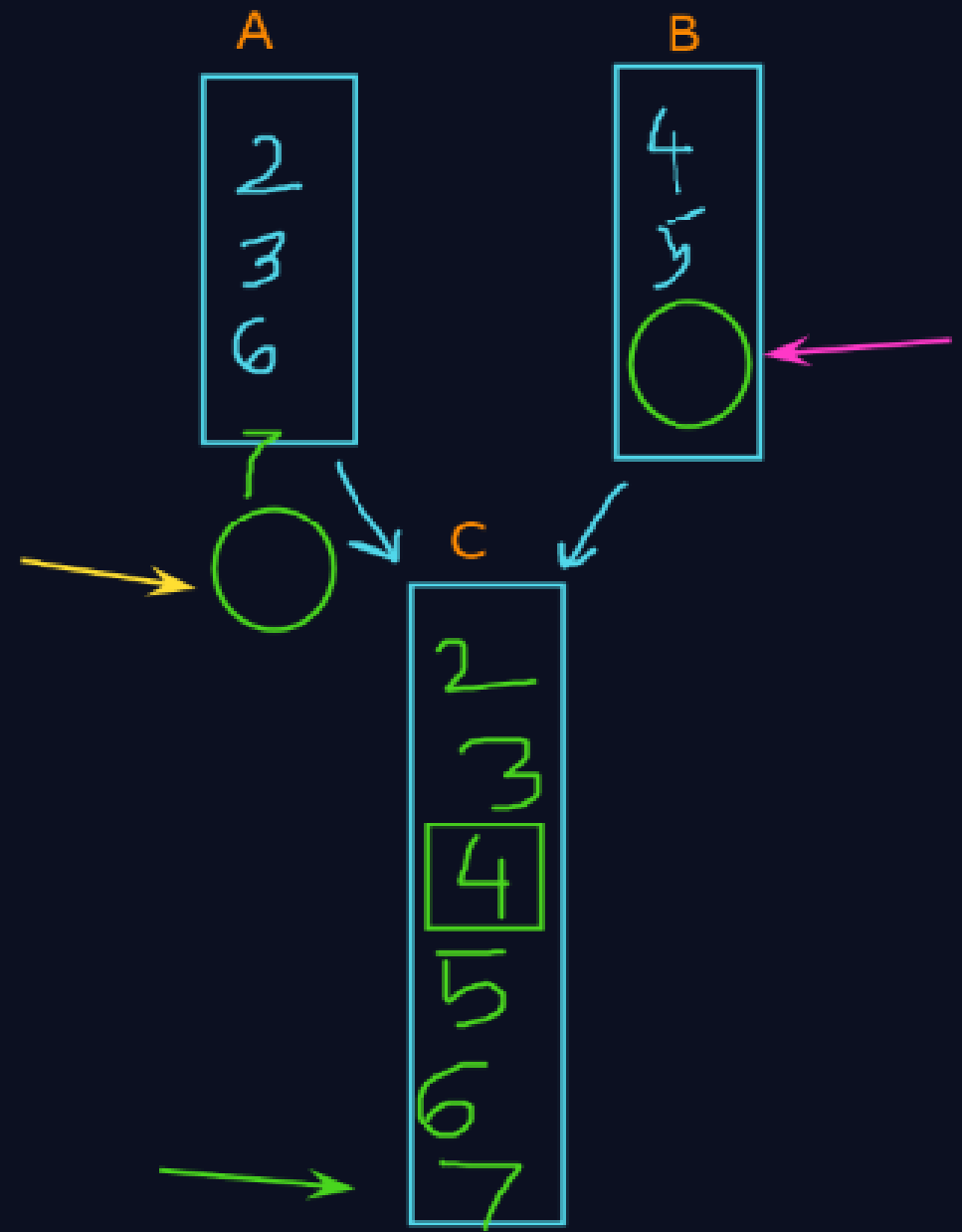
```
Merge (A, p, q, r)
1   n1 ← (q - p) + 1
2   n2 ← (r - q)
3   create arrays L[1..n1+1] and R[1..n2+1]
4   for i ← 1 to n1 do
5       L[i] ← A[(p + i) - 1]
6   for j ← 1 to n2 do
7       R[j] ← A[q + j]
8   L[n1 + 1] ← ∞
9   R[n2 + 1] ← ∞
10  i ← 1
11  j ← 1
12  for k ← p to r do
12.5    count ← count + 1
13      if L[i] ≤ R[j]
14          then A[k] ← L[i]
15              i ← i + 1
16      else A[k] ← R[j]
17          j ← j + 1
```

```
int i=0, j=0;  
int k=1;
```

```
while(i<n1 && j<n2){  
    if(L[i] <= R[j])  
    {  
        arr[k] = L[i];  
        i++;  
    }else{  
        arr[k] = R[j];  
        j++;  
    }  
    k++;  
}
```

```
while(i < n1){  
    arr[k] = L[i];  
    i++;  
    k++;  
}
```

```
while(j < n2){  
    arr[k] = R[j];  
    j++;  
    k++;  
}
```



## Day 10: Algorithms and Data Structures

Date : 4-April-2025

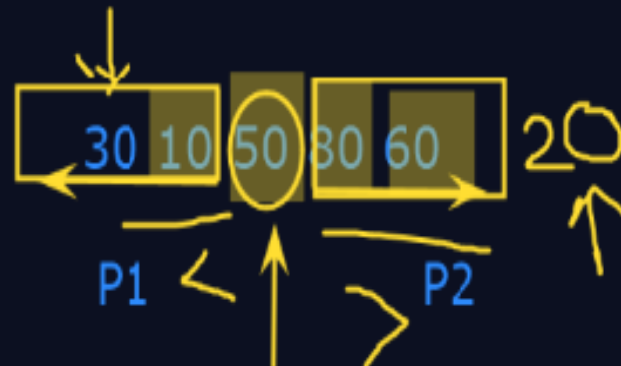
You are screen sharing

Stop share

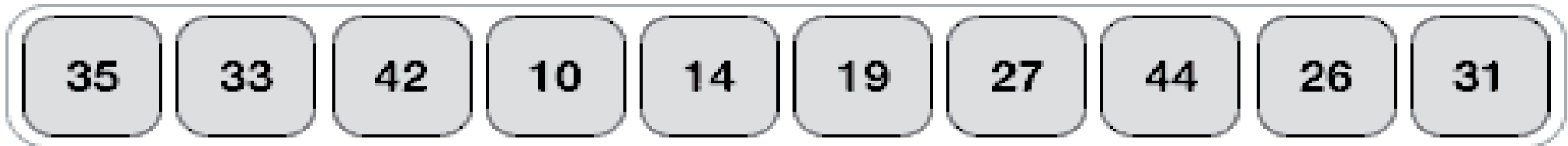
### Topics:

- Quick Sort
- Queue
- Circular Queue
- Hashing

Pivot



## Unsorted Array



#### 4. Merge sort:

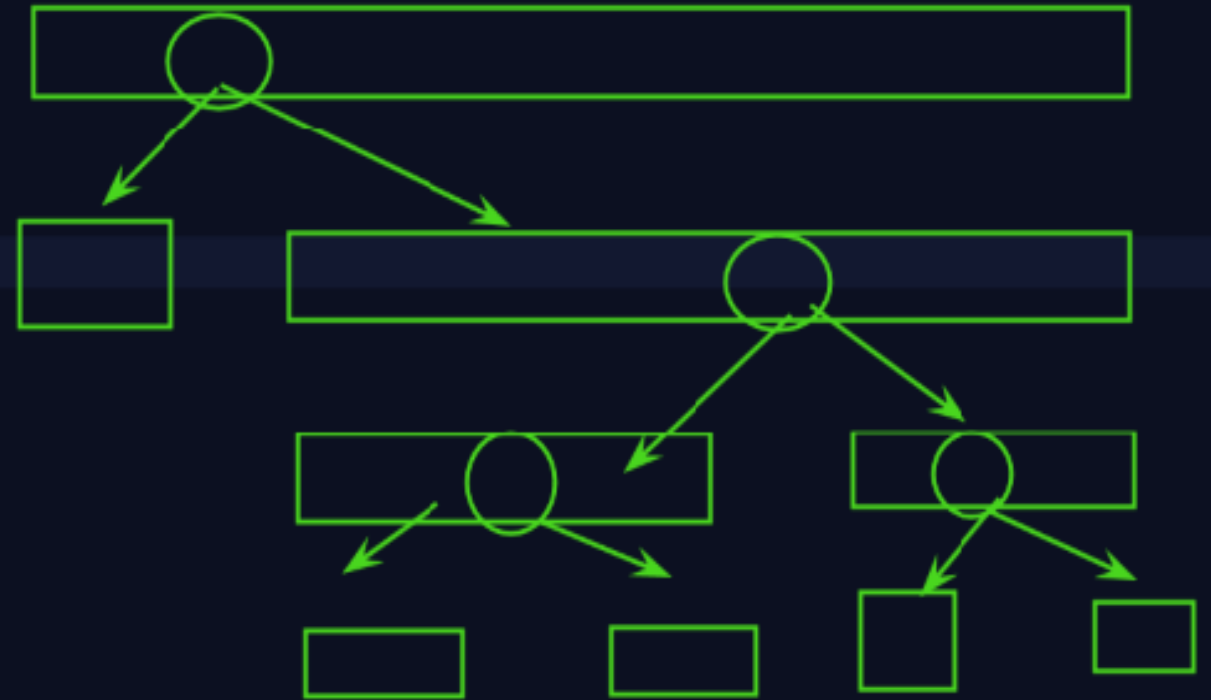
- Divide and Conquer Algorithm strategy

#### 5. Quick sort:

- Divide and Conquer Algorithm strategy
- use pivote element

A = 10 40 50 80 30  
B = 60 40 50 80 30 90  
C = 10 40 50 80 90  
D = 10 50 30 80 90

↑  
Pivot





The following procedure implements quicksort:

QUICKSORT( $A, p, r$ )

```
1  if  $p < r$ 
2       $q = \text{PARTITION}(A, p, r)$ 
3      QUICKSORT( $A, p, q - 1$ )
4      QUICKSORT( $A, q + 1, r$ )
```

To sort an entire array  $A$ , the initial call is QUICKSORT( $A, 1, A.length$ ).

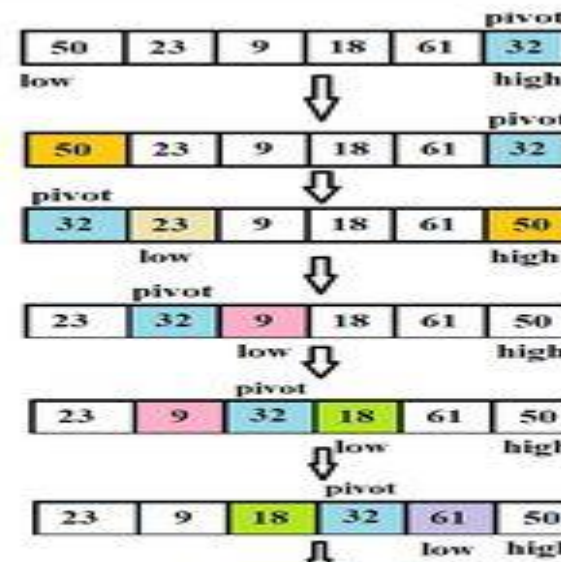
### Partitioning the array

The key to the algorithm is the PARTITION procedure, which rearranges the subarray  $A[p \dots r]$  in place.

PARTITION( $A, p, r$ )

```
1   $x = A[r]$ 
2   $i = p - 1$ 
3  for  $j = p$  to  $r - 1$ 
4      if  $A[j] \leq x$ 
5           $i = i + 1$ 
6          exchange  $A[i]$  with  $A[j]$ 
7  exchange  $A[i + 1]$  with  $A[r]$ 
8  return  $i + 1$ 
```

# Quick Sort



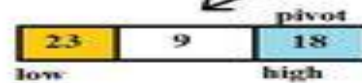
```

if arr[low] > arr[pivot]:
    swap(arr[low], arr[pivot])
    low++;
else:
    continue;

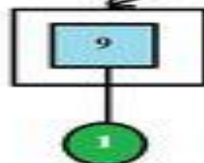
```



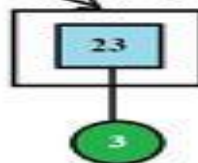
Quick Sort on Left side



Quick Sort on Left side



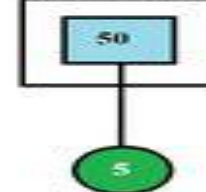
Quick Sort on Right side



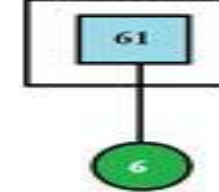
Quick Sort on Right side



Quick Sort on Left side



Quick Sort on Right side



Final Sorted Array:



```
int pi = partition(arr, low, high),
```

```
quicksort(arr, low, pi-1);
```

```
quicksort(arr, pi+1, high);
```

```
}
```

```
}
```

```
static int partition(int arr[], int low, int high){
```

```
int pivot = arr[high];
```

```
int i = (low-1);
```

```
for(int j=low;j<=high-1;j++){
```

```
    if(arr[j] < pivot){
```

```
        i++;
```

```
        swap(arr, i, j);
```

```
    }
```

```
}
```

```
swap(arr, i+1, high); //Correct position of pivot element
```

```
return (i+1); //index of pivot element
```

```
}
```

44, 77, 22, 99, 11, 55, 66, 33

22 77 44 99 11 55 66 33

22 11 44 99 77 55 66 33

22 11 33 99 77 55 66 44

low

high

11 22

44 77 55 66 99

55 77 66 99

55 66 77 99

Pooja Nalawade raised hand View ×

```
static void quicksort(int arr[], int low, int high){
```

```
    if(low < high)
```

```
    {
        int pi = partition(arr, low, high);
```

```
        quicksort(arr, low, pi-1); //Left array : P1
```

```
        quicksort(arr, pi+1, high); //Right array : P2
```

```
    }
```

```
static int partition(int arr[], int low, int high){
```

```
    int pivot = arr[high];
```

```
    int i = low-1;
```

```
    for(int j=low; j<=high-1; j++){
```

```
        if(arr[j] < pivot)
```

```
        {
```

```
            i++;
```

```
            swap(arr, i, j);
```

```
        }
```

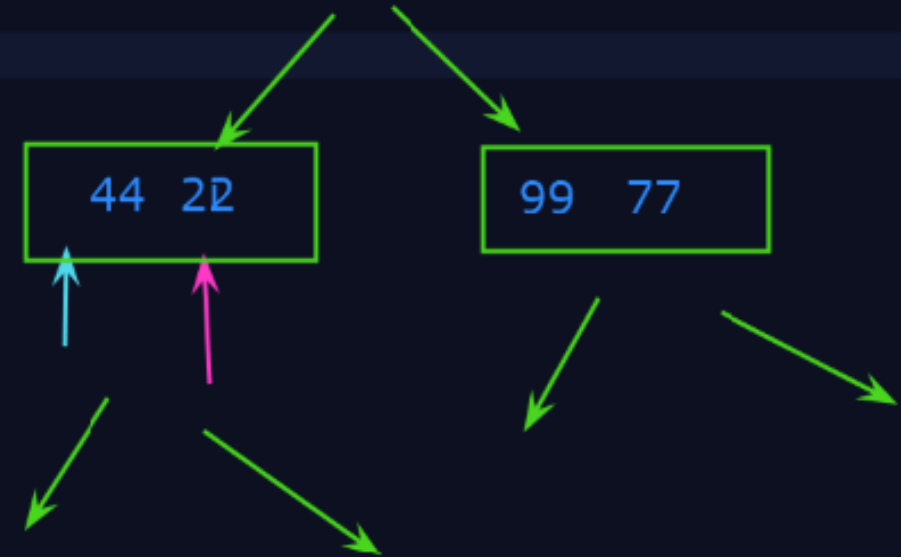
```
    }
```

```
    swap(arr, i+1, high);
```

```
    return(i+1);
```

```
}
```

44, 22, 55, 99, 77



# Heap

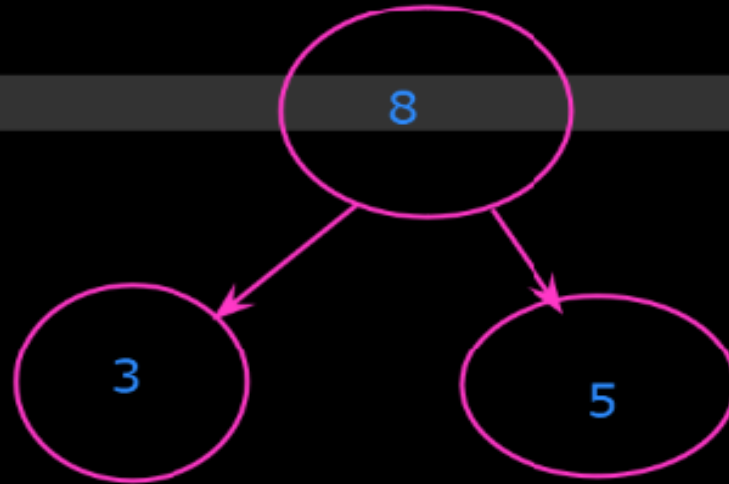
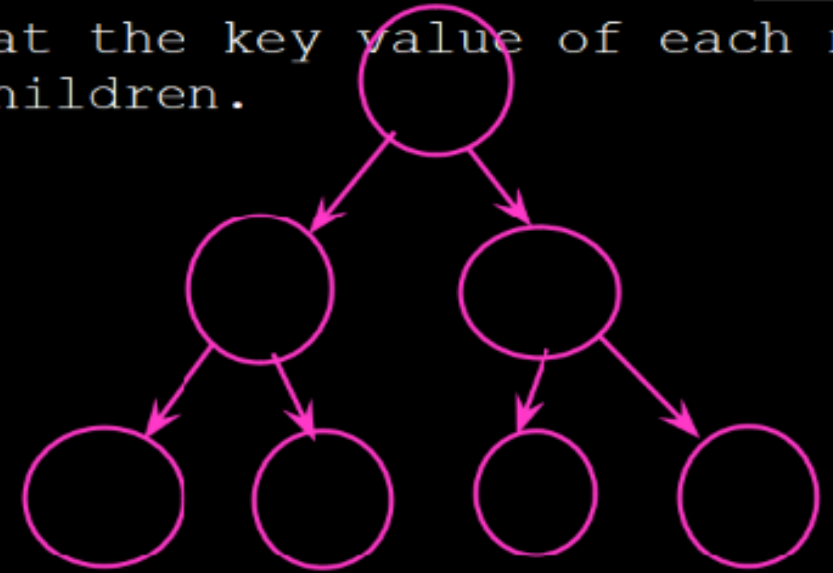


## Definition:

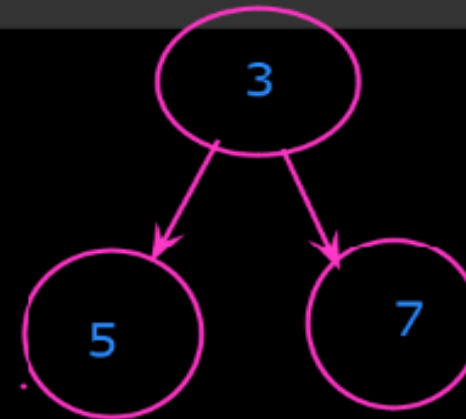
-a special form of complete binary tree that the key value of each node is smaller (larger) than the key value of its children.

## Types of heap:

1. Max-Heap: root node has largest value
2. Min-Heap: root node has smallest value



Max Heap



Min Heap

Heap

# Heap

- **Definition in Data Structure**

- **Heap:** A special form of **complete binary tree** that key value of each node is no smaller (larger) than the key value of its children (if any).

- **Max-Heap: root node has the largest key.**

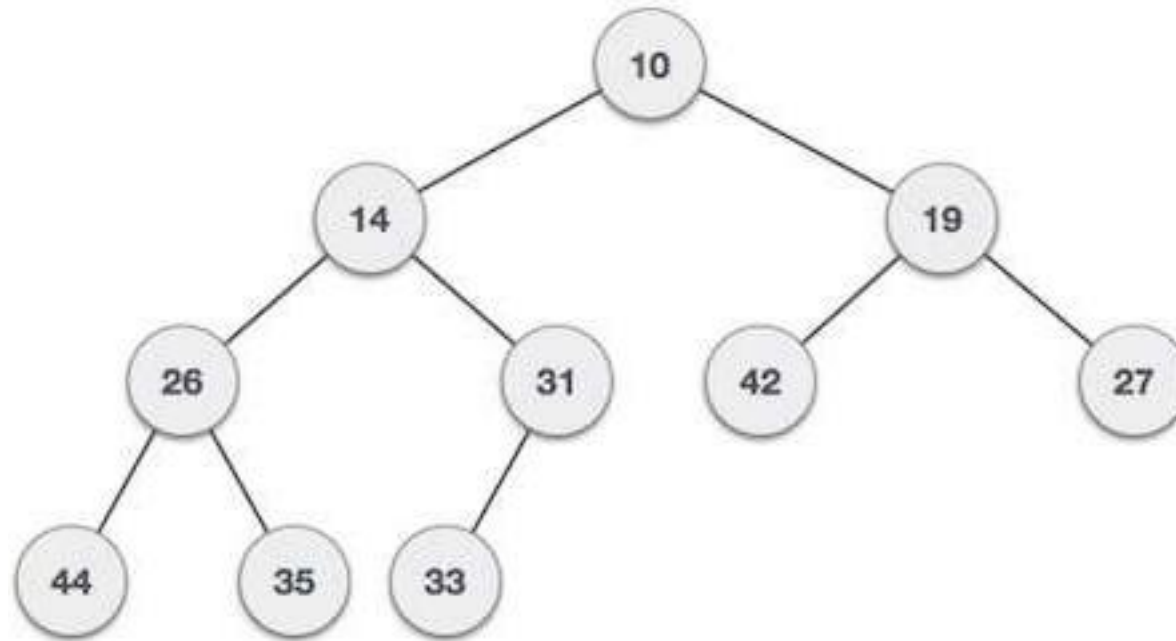
- A **max tree** is a tree in which the key value in each node is **no smaller than** the key values in its children.
- A **max heap** is a **complete binary tree** that is also a max tree.

- **Min-Heap: root node has the smallest key.**

- A **min tree** is a tree in which the key value in each node is **no larger than** the key values in its children.
- A **min heap** is a **complete binary tree** that is also a min tree.

# Heap

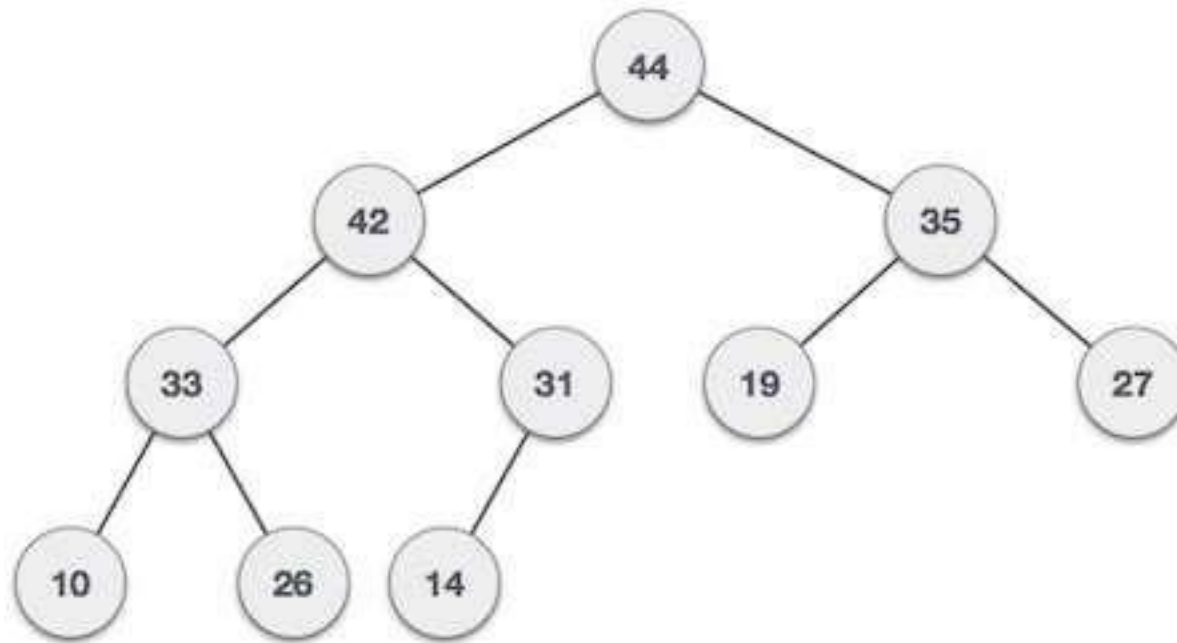
- Min-Heap
  - Where the value of the root node is less than or equal to either of its children
  - For input 35 33 42 10 14 19 27 44 26 31





# Heap

- Max-Heap –
  - where the value of root node is greater than or equal to either of its children.
  - For input 35 33 42 10 14 19 27 44 26 31



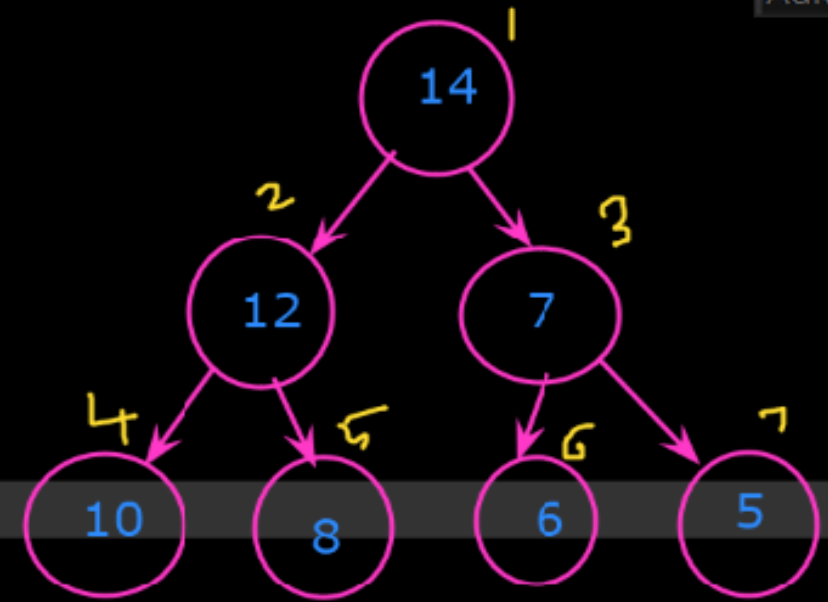
Types of heap:

1. Max-Heap: root node has largest value
2. Min-Heap: root node has smallest value

$$\text{Parent} = i/2$$

$$\text{Lc} = 2i$$

$$\text{RC} = 2i+1$$



Max heap

14 12 7 10 8 6 5

1 2 3 4 5 6 7

Types of heap:

1. Max-Heap: root node has largest value
2. Min-Heap: root node has smallest value

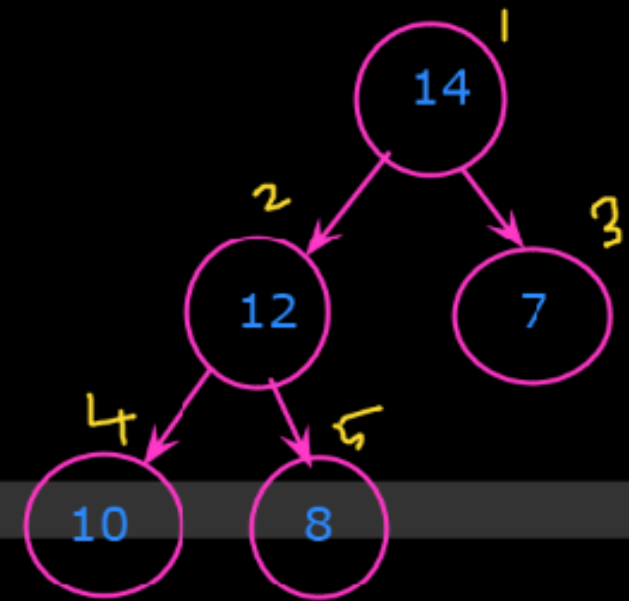
$$\text{Parent} = i/2$$

$$\text{Lc} = 2i$$

$$\text{RC} = 2i+1$$

14 12 7 10 8

1 2 3 4 5 6 7



Max heap

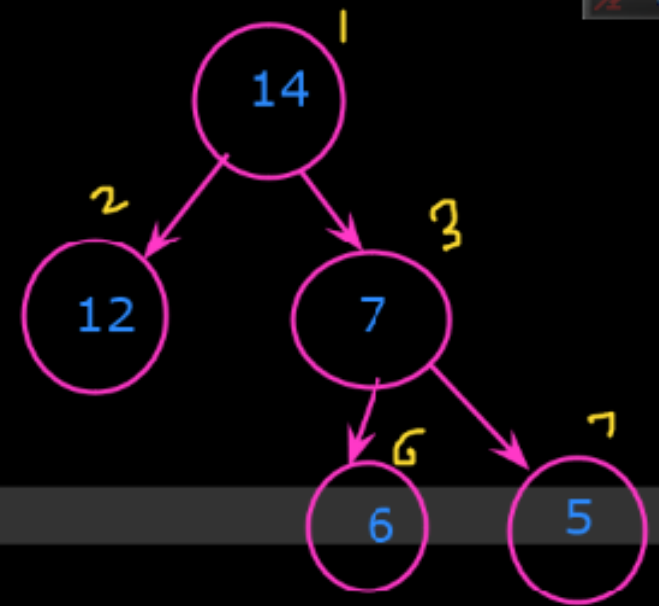
Types of heap:

1. Max-Heap: root node has largest value
2. Min-Heap: root node has smallest value

$$\text{Parent} = i/2$$

$$\text{Lc} = 2i$$

$$\text{RC} = 2i+1$$



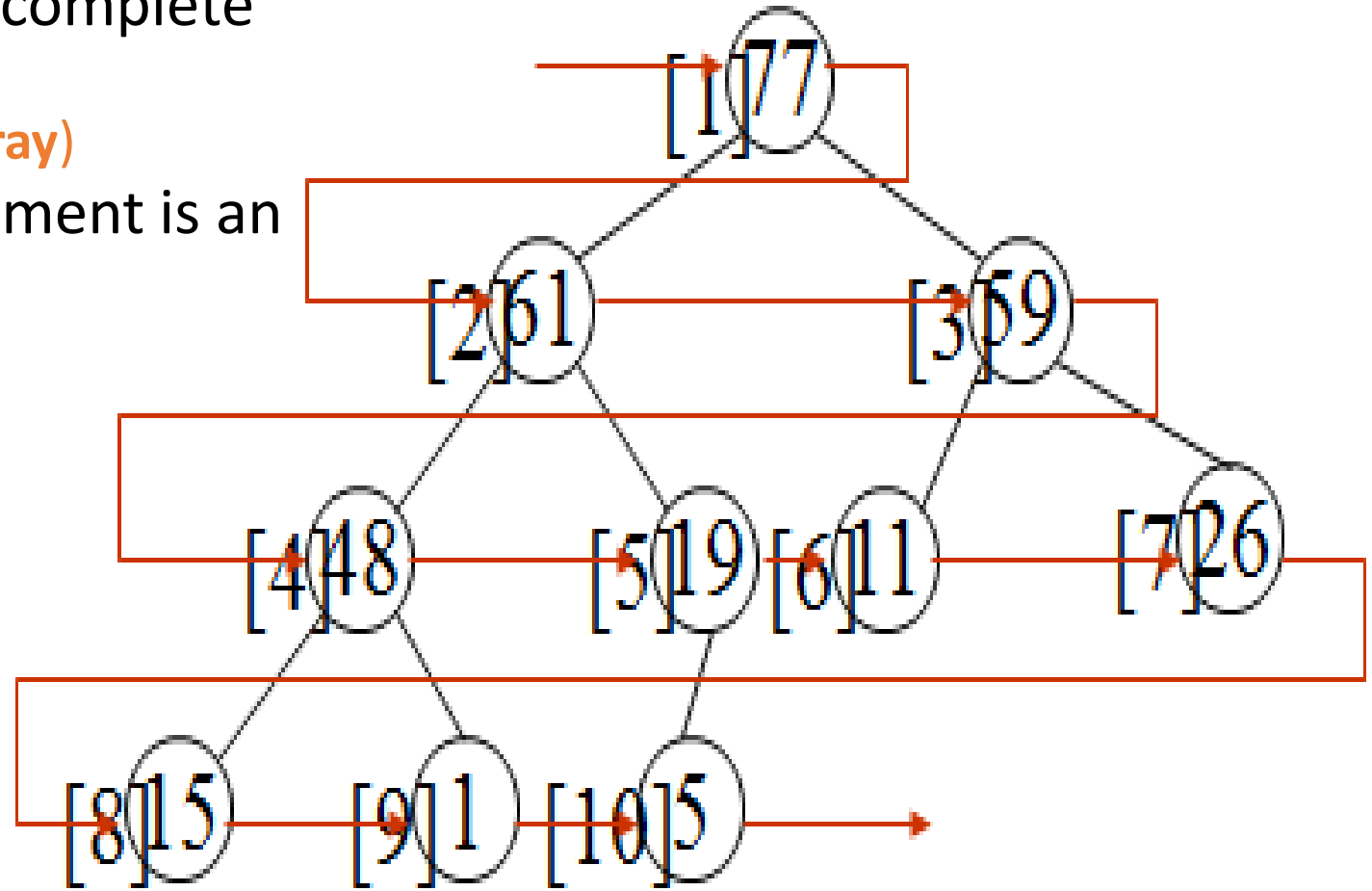
Max heap

14 12 7 - - 6 5

1 2 3 4 5 6 7

- **Note:**

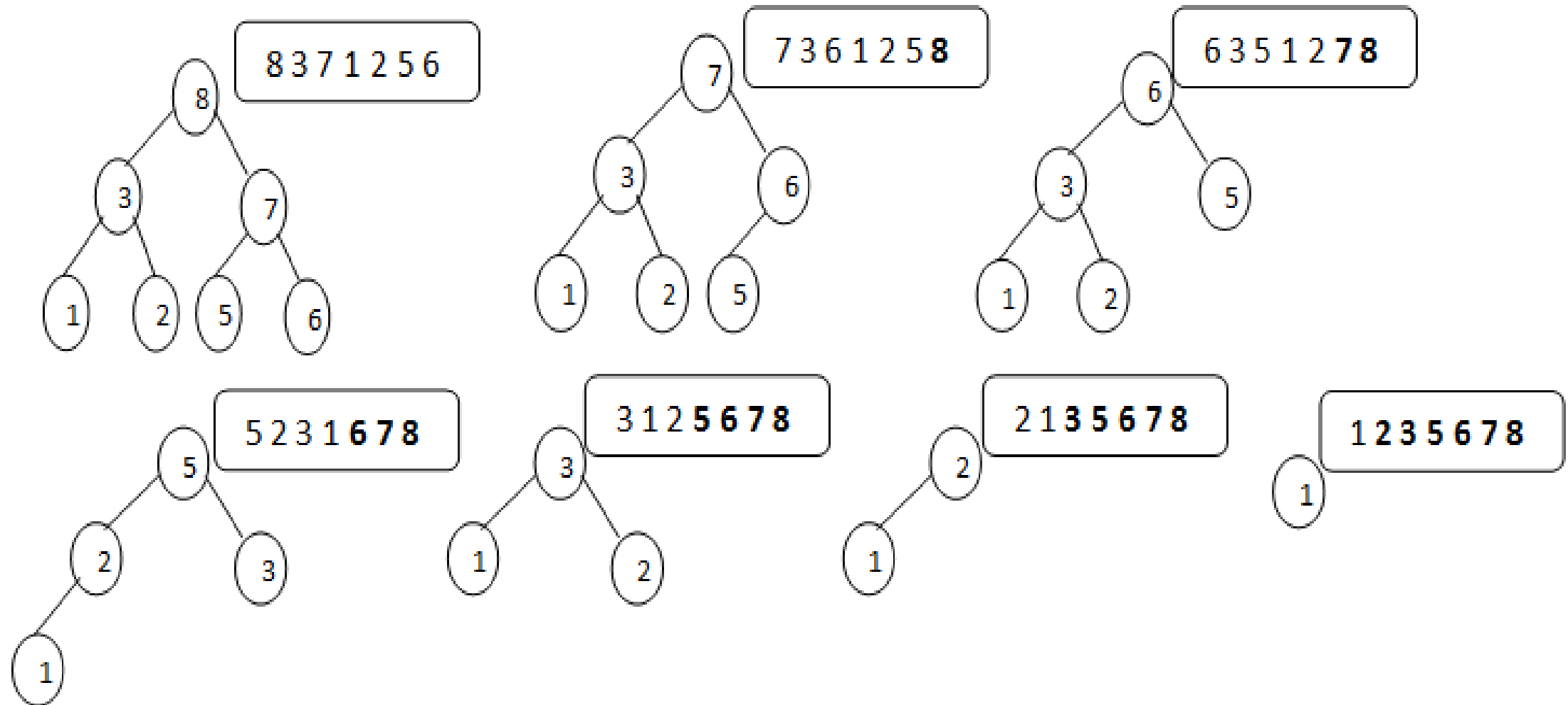
- Heap in data structure is a complete binary tree!
  - (Nice representation in Array)
- Heap in C program environment is an array of memory.



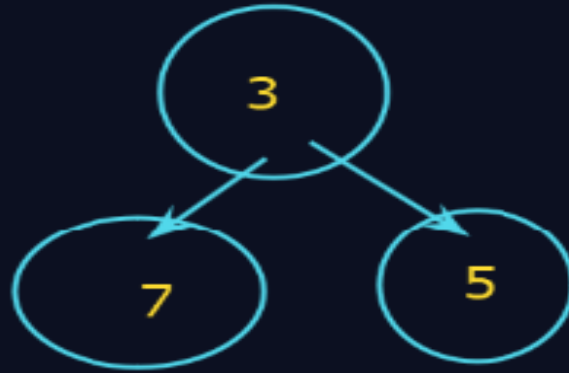
— Stored using array in C

|       |    |    |    |    |    |    |    |    |   |    |
|-------|----|----|----|----|----|----|----|----|---|----|
| index | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9 | 10 |
| value | 77 | 61 | 59 | 48 | 19 | 11 | 26 | 15 | 1 | 5  |

**Example:-** The fig. shows steps of heap-sort for list (2 3 7 1 8 5 6)



HEap Sort:



Min Heap



Max Heap

Heap:

-A special form of complete binary tree such that the key value of each is either smaller or greater than the root node.

Types of heap:

1. Max heap:

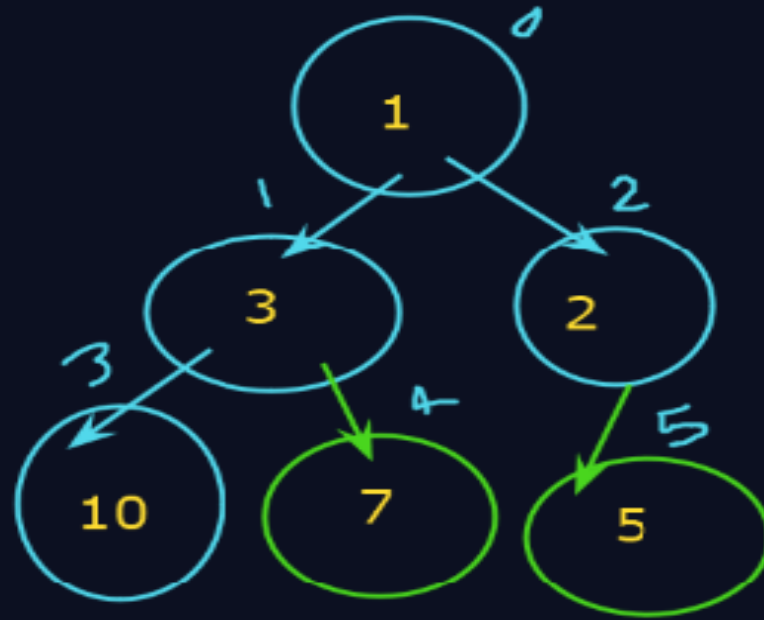
-A max heap in which the key value of the root node is greater than the other nodes.

2. Min heap:

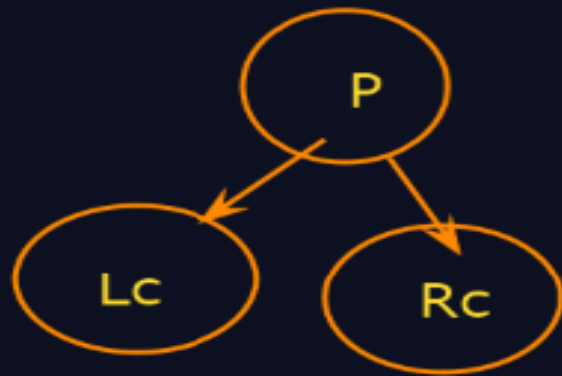
-A min heap in which the key value of the root node is smaller than the other nodes.

Heap

Min Heap



Max Heap

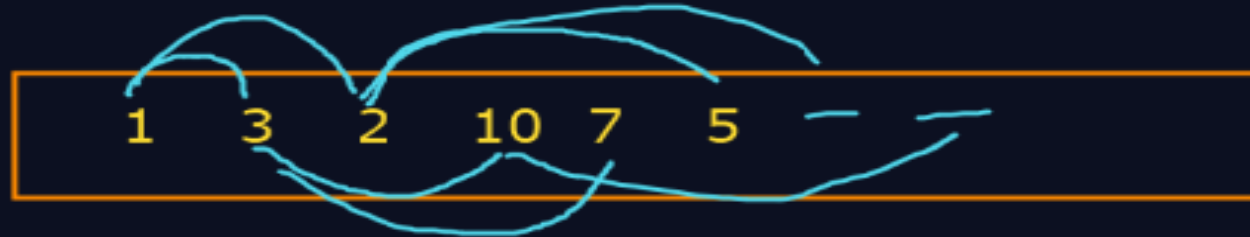


$i = 0$

Parent :  $i$   
 LC :  $2*i + 1 = 1$   
 RC :  $2*i + 2 = 2$

$i = 1$

Parent :  $i/2$   
 LC :  $2*i$   
 RC :  $2*i + 1$





```

class Hsort{

    void heapify(int arr[],int n, int i){

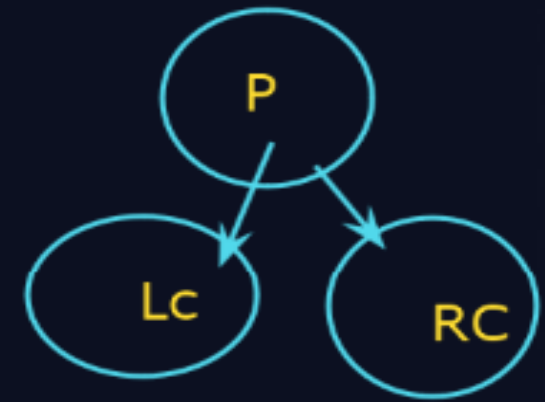
        int largest = i; //Parent
        int l = 2*i+1; //LC
        int r = 2*i+2; //RC

        if(l < n && arr[l] > arr[largest])
            largest = l;
        if(r < n && arr[r] > arr[largest])
            largest = r;

        if( largest != i){
            int temp = arr[i];
            arr[i] = arr[largest];
            arr[largest] = temp;

            heapify(arr, n, largest);
        }
    }
}

```



# HEAP SORT

| Best          | Average       | Worst         |
|---------------|---------------|---------------|
| $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ |



Array



Recursion



Binary Heap

**sort (A)**

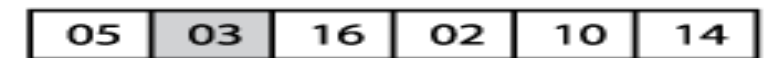
1. **buildHeap(A)**
2. **for**  $i = n - 1$  **downto** 1 **do**
3.     swap  $A[0]$  with  $A[i]$
4.     **heapify** (A, 0, i)
- end**

**buildHeap (A)**

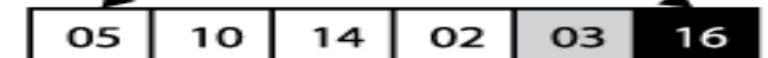
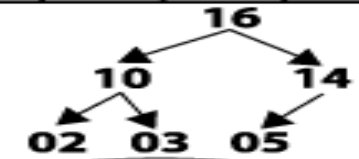
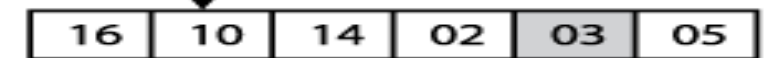
1. **for**  $i = \lfloor n/2 \rfloor - 1$  **downto** 0 **do**
2.     **heapify** (A, i, n)
- end**

**heapify (A, idx, max)**

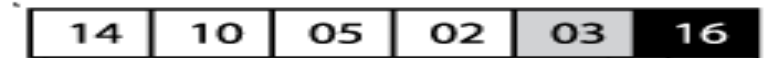
1.      $\text{left} = 2 * \text{idx} + 1$
2.      $\text{right} = 2 * \text{idx} + 2$
3.     **if** ( $\text{left} < \text{max}$  **and**  $A[\text{left}] > A[\text{idx}]$ ) **then**
4.          $\text{largest} = \text{left}$
5.     **else**  $\text{largest} = \text{idx}$
6.     **if** ( $\text{right} < \text{max}$  **and**  $A[\text{right}] > A[\text{largest}]$ ) **then**
7.          $\text{largest} = \text{right}$
8.     **if** ( $\text{largest} \neq \text{idx}$ ) **then**
9.         swap  $A[i]$  and  $A[\text{largest}]$
10.     **heapify** (A, largest, max)
- end**



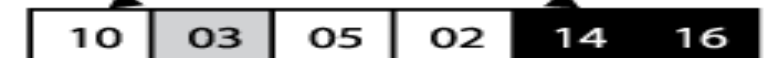
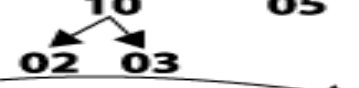
**buildHeap**



*Might no longer be a heap*     *Sorted sub-array*



*A heap again*     *Sorted sub-array*



*A heap again*     *Sorted sub-array*

